BMEN 5251 | Final Project

Problem Statement:
I am solving a drug delivery of a topical cream such as 0.1% tacrolimus on skin. I am using Fick's 2nd Law of Diffusion as the governing equation in two dimensions $\frac{\delta c}{\delta t} = D(\frac{\delta^2 c}{\delta x^2} + \frac{\delta^2 c}{\delta y^2})$

Which defines how the concentration c changes with respect to time t over a boundary when not at steady state. For drug diffusion across the skin, this non-steady state version of Fick's Law is especially relevant.

To determine the optimal dosing of 0.1% tacrolimus on the skin, it is pertinent to determine the half life of the drug concentration and time it takes for the drug to go from the transdermal layer into the medication dermis.

My hypothesis is that while the topical cream is applied on the surface of the skin with a specific surface area, as the drug diffuses into the skin, the drug can expand further than the original surface area as it diffuses from the transdermal layer into the medically active dermis. Thus, the proposed region of interest and associated boundary conditions are as follows:

Sides: (Neumann Boundary)
$-D(n \cdot \nabla c) = 0 \rightarrow$ no flux natural boundary

Bottom Boundary (Dirichlet Boundary):
$c(x, y, 0) = 0$

Top Boundary (Dierchlet Boundary):
$c(x, y, 0) = c_0 = 0.04\, mg$

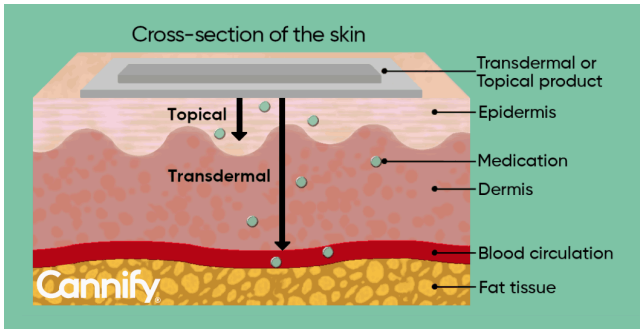
To verify my model, I will have the FEM model calculate the time it takes for half of the concentration to travel down the region of interest and see if that aligns with the equation $t_{1/2}=\ln(2)/k$

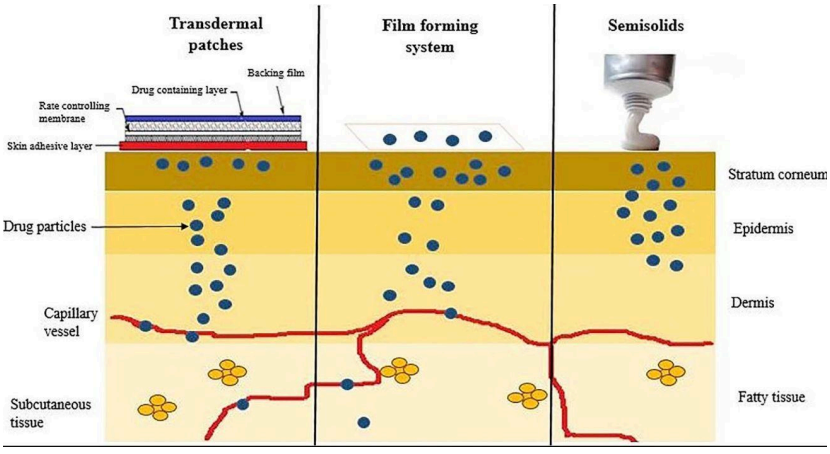
Assumptions made:

- All skin has the same diffusion coefficient which is equivalent to the stratum corneum (D= 1E-8 cm^2/hour Partition coefficient and diffusion coefficient determinations of 50 compounds in human intact skin, isolated skin layers and isolated stratum corneum lipids - ScienceDirect)
- Tacrolimus is a large molecule in terms of skin penetration, therefore when defining the parameters for diffusion, it was treated as such (Tacrolimus: Physicochemical stability challenges, analytical methods, and new formulations - ScienceDirect)
- Clinical pharmacokinetics describes the rate constant to be k=0.0105 [1/hr] A Multicenter Study of the Pharmacokinetics of Tacrolimus Ointment after First and Repeated

Application to Children with Atopic Dermatitis - Journal of Investigative Dermatology based on the half life being 66 +/- 27 hours.

- For simplicity's sake, I did not account for the different ways drugs absorb vertically (ie do they penetrate transcellularly or intercellularly.) So there's no flux at the two side boundaries and the boundaries are all Dierchlet
- Tacrolimus is also a multicompartment model where absorption, metabolism, and elimination are all factors that affect concentration. For time and simplicity sake, I will keep the model as a single compartment for absorption.
- Tacrolimus exhibits a small amount of lateral diffusion
- We assumed the region of interest for diffusion is the transdermal layer all the way into the dermis which is about 3.5mm. The dermis is considered the medically relevant layer where drug delivery can be bound or metabolized. I did not take into account how each of the different layers may have their own diffusion coefficients, however an extension of this could be to create a FEM model for each individual layer and then superimpose them all on top of each other.





FEM Code Explanation

MyMesher.m → Used to create the triangular mesh of my region. This is the Mesher code from Prof. Barocas provided in HW8. I have included a copy of it as part of my submission with comments to demonstrate understanding of the code I am using

FinalProject.m

- Set the parameters of the problem so that the model is physiologically relevant to skin diffusion
- Define the corners of the region of interest for MyMesher.m and setting the Dierchlet boundaries
- Generate and plot the mesh
- Define the Gauss Points
- Basis functions of phi_i, set up derivatives dfds and dfde
- FEM Code → adapted from HW8
    - For each element, we loop through the gauss points and assess each local node
    - We define the global positioning of the global node using the local node and element from the transition matrix (ig = T(e,i);
    - Lines 95-99: Initialize how physical position changes with respect to the computational space in both the x and y directions
    - Compute jacobian
    - Lines 102-105: Loop through each local node, set up map back to the global node using T. Compute dfidx and dfidy
    - Lines 106-111: Loop through each local node, set up map back to global node using T. Compute dfjdx and dfjdy for the approximations of concentration.
    - Construct mass matrix and stiffness matrix. No load vector needed
    - Yields dc/dt → neds od15 to find actual concentration c
- Use DB array to define which sides of the region of interest have a Dierchlet boundary enforced
- Reconstruct initial condition vector with the boundary conditions
- Use ode15 to solve for c
- Compare half life of FEM model with analytical solution $t_{1/2} = ln(2)/k$ to validate model
- Plot the heat maps at time =0 and time at tFinal = 72 hours of the concentration

Governing Equation (Fick's 2nd Law, not steady state)

$$\frac{\partial C}{\partial t} = D\left(\frac{\partial^2 C}{\partial x^2} + \frac{\partial^2 C}{\partial y^2}\right)$$

D = Diffusion coefficient
Let the concentration C @ the top have a time dependency

$C(x,y,t) = C_0 e^{-Kt}$
C = concentration
C_0 = initial concentration of drug applied on the skin = 0.04mg
K = rate constant [1/hr] = 5 1/hr
t = time [1/hr]

BOUNDARY CONDITIONS
Sides (Neumann)
$-D(n \cdot \nabla C) = 0$ flux is 0 in the $\perp$ direction

Bottom Boundary: (Dirchlet)
$C(x,y,0) = 0$

TOP Boundary (Dirchlet)
$C(t=0) = C_0$
$C(x,y,t) = C_0 e^{-Kt}$ → concentration decays over time due to evaporation, absorption into tissue, etc.

Parameters calculation

$$D = \frac{1E\text{-}8\ cm^2}{hr} \cdot \frac{1E\text{-}2m}{cm} \cdot \frac{1E\text{-}2m}{cm} \cdot \frac{\mu m}{1E\text{-}6m} \cdot \frac{\mu m}{1E\text{-}6m}$$

$\boxed{D = 1\ \mu m^2/hr}$

C_0 = 0.1% ointment = $\frac{1mg\ tacrolimus}{1000mg\ cream}$

Surface area of application: 2 x 2 mm = $0.04\ cm^2$
thin layer conc = $1.5mg/cm^2$

amt of ointment: $0.04\ cm^2 \cdot \frac{1mg}{cm^2} = 0.04$ mg ointment

amt. of tacrolimus: $0.04\ mg\ ointment \cdot \frac{1mg\ tacrolimus}{1000mg\ ointment} = \boxed{0.04mg\ tacrolimus}$

$\boxed{C_0 = 0.04\ mg}$

Half-life of 1st order : $t_{1/2} = \frac{\ln(2)}{K}$
K = 0.0105 1/hr
expected $t_{1/2} = \frac{\ln(2)}{0.0105} = 66.014$ hrs

Finding the weak form:

$$\frac{\partial c}{\partial t} = D\left(\frac{\partial^2 c}{\partial x^2} + \frac{\partial^2 c}{\partial y^2}\right) = D\nabla^2 c \qquad \frac{\partial c}{\partial t} = D\nabla^2 c \Rightarrow \frac{\partial c}{\partial t} - D\nabla^2 c = 0$$

Multiply test fxn and integrate

$$\int \frac{\partial c}{\partial t}\psi_i \, dA = \int D\nabla^2 c \psi_i \, dA$$

$$= D\int \nabla\psi_i \cdot \nabla c \, dA + D\int \psi_i \nabla^2 c \, dA = \oint_{\partial A} (n\cdot\nabla c)\, ds$$

$$\int \frac{\partial c}{\partial t}\psi_i \, dA = D\int \nabla\psi_i \cdot \nabla c \, dA + D\int \psi_i \nabla^2 c \, dA = \oint_{\partial A} (n\cdot\nabla c)\, ds$$

$$\int \frac{\partial c}{\partial t}\psi_i \, dA - \int D\nabla^2 c \psi_i \, dA = \int \frac{\partial c}{\partial t}\psi_i \, dA - \left(D\int \nabla\psi_i \cdot \nabla c \, dA + D\int \psi_i \nabla^2 c \, dA\right) = \int \frac{\partial c}{\partial t}\psi_i \, dA - \oint_{\partial A} (n\cdot\nabla c)\psi_i \, ds$$

product rule — DVT

$$\int \frac{\partial c}{\partial t}\psi_i \, dA - \left(D\int \nabla\psi_i \nabla c \, dA + D\int \psi_i \nabla^2 c \, dA\right) = \int \frac{\partial c}{\partial t}\psi_i \, dA - \oint_{\partial A} (n\cdot\nabla c)\psi_i \, ds$$

0 b/c no flux

$$\int \psi_i \frac{dc}{dt} \, dA + \int D\nabla\psi_i \cdot \nabla c \, dA = 0$$

$$\nabla c = \begin{bmatrix} \frac{\partial c}{\partial x} \\ \frac{\partial c}{\partial y} \end{bmatrix}, \qquad \nabla\psi_i = \begin{bmatrix} \frac{\partial \psi_i}{\partial x} \\ \frac{\partial \psi_i}{\partial y} \end{bmatrix}$$

$$\nabla c \cdot n = \sum_{j=1}^{N} c_j \left(\frac{\partial \psi_j}{\partial x}\cdot n_x + \frac{\partial \psi_j}{\partial y}\cdot n_y\right)$$

$$\int_A \psi_i \frac{\partial c}{\partial t} \, dA + \int_A D\left(\frac{\partial \psi_i}{\partial x}\frac{\partial c}{\partial x} + \frac{\partial \psi_i}{\partial y}\frac{\partial c}{\partial y}\right) dA = 0$$

$$\int_A D\left(\frac{\partial \psi_i}{\partial x}\frac{\partial c}{\partial x} + \frac{\partial \psi_i}{\partial y}\frac{\partial c}{\partial y}\right) dA = -\int_A \psi_i \frac{\partial c}{\partial t} \, dA$$

$$\int D\left(\frac{\partial \psi_i}{\partial x}\frac{\partial c}{\partial x} + \frac{\partial \psi_i}{\partial y}\frac{\partial c}{\partial y}\right) dA + \int \psi_i \frac{\partial c}{\partial t} \, dA = 0$$

Discretize using $C = \sum C_j \psi_j$

$$\sum_{j=1}^{N} C_j \int_D \underbrace{\left(\frac{\partial \psi_i}{\partial x}\frac{\partial \psi_j}{\partial x} + \frac{\partial \psi_i}{\partial y}\frac{\partial \psi_j}{\partial y} \right)}_{\text{stiffness } A} dA + \sum_{j=1}^{N} \frac{dC_j}{dt} \underbrace{\int \psi_i \psi_j \, dA}_{\text{mass matrix } M} = 0$$

WEAK FORM
* FEM code adapted from HW #6

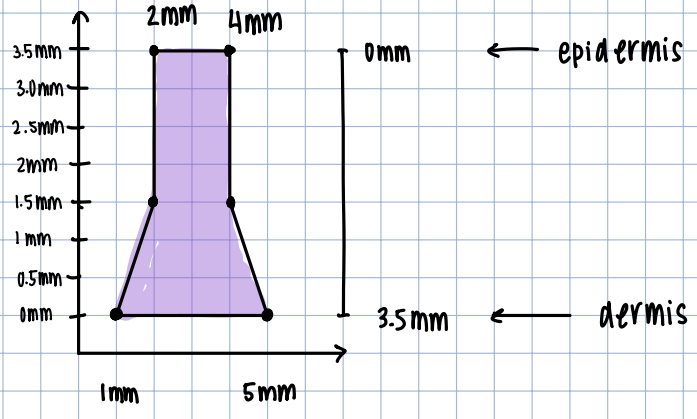
$$AC + M \cdot \frac{dC}{dt} = 0$$

$$\frac{dC}{dt} = M \backslash (-A * C)$$ → input for ode15 to solve time derivative

$$\frac{dx}{dt} = f(x)$$ (x-vector)

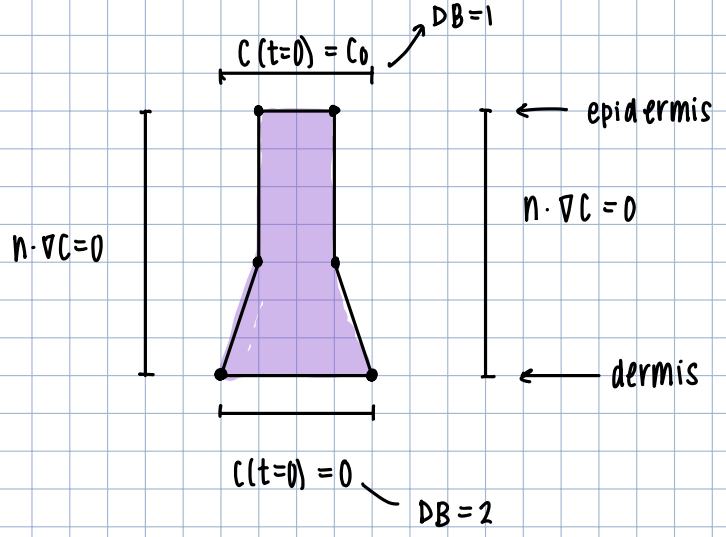
ode15s(fh, time range, initial)
function [dxdt] = MyFunc(x)
* from Midterm Project notes

Region of Interest:
polygonize using Barocas' MyMesher.m



Coordinates of boundary corner:
(2, 0)
(4, 0)
(2, -2)
(4, -2)
(1, -3.5)
(5, -3.5)

px = [2, 4, 2, 4, 1, 5]
py = [0, 0, -2, -2, -3.5, -3.5]



Finding the 1/2 life from the FEM model:
- ode15 gives discretized (t, C) values for each node
- we want to find the exact moment where $C = \frac{1}{2} C_0$ of the top boundary which is btwn 2 nodes: (t_1, C_1) and (t_2, C_2)

$$t_{1/2} = t_1 + \frac{\frac{1}{2}C_0 - C_0}{C_2 - C_1}(t_2 - t_1) \Longrightarrow 66.0973 \text{ hrs (from code)}$$

Compare w/ analytical: $t_{1/2} = \frac{\ln(2)}{K} = 66.014$ hours

} relatively similar ✓

Table of Contents

..... 1
BMEN 5251 | Final Project 1
FEM Code 3
Comparison of analytical and FEM values 5
Plots 5

```
close all; clear; clc;
warning('off','all'); % i put this because i think MyMesher was
% causing problems with publishing the code as a pdf
```

BMEN 5251 | Final Project

Katriana Trinh May 11th, 2026

```
% Parameters
D = 1;  % diffusion coefficient, um^2/hr
c0 = 0.04;% initial top concentration, mg
k = 0.0105;% decay constant, 1/hr
tFinal = 72;% final time, hr


% Region of interest
px = [2; 4; 4; 5; 1; 2]; % mm
py = [3.5; 3.5; 1.5; 0; 0; 1.5];  % mm

NNT = 500; % # of triangular meshes

% Dirichlet boundaries mapping for MyMesher
DB = [
    2  3.5   4  3.5;   % top boundary
    1  0     5  0  % bottom boundary
];

% Generate mesh using Barocas code MyMesher.m
[T, xp, yp, Dirichlet] = MyMesher(px, py, NNT, DB);

% Plot mesh
figure;
triplot(T,xp,yp);
axis equal;
title('Skin Mesh');
xlabel('x (mm)');
ylabel('y (mm)');

% Following code is all adapted and adjusted from HW8
% Gauss points
gps = [1 4 1]/6;
gpe = [1 1 4]/6;
```

```
gw  = [1 1 1]/6;

NG = 3;
NL = 3;

NN = length(xp);
NE = size(T,1);

% Shape functions
f = zeros(NL,NG);

for g = 1:NG
    f(1,g) = 1 - gps(g) - gpe(g);
    f(2,g) = gps(g);
    f(3,g) = gpe(g);
end

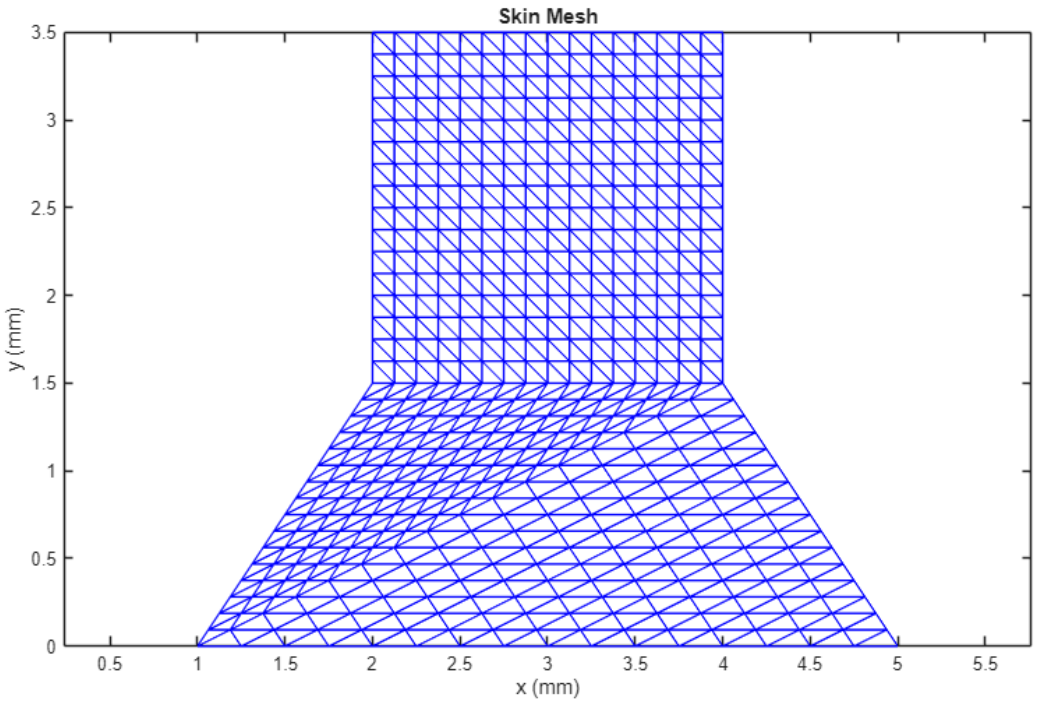
% Derivatives wrt xi
dfds = zeros(NL,NG);

for g = 1:NG
    dfds(1,g) = -1;
    dfds(2,g) =  1;
    dfds(3,g) =  0;
end

% Derivatives wrt eta
dfde = zeros(NL,NG);

for g = 1:NG
    dfde(1,g) = -1;
    dfde(2,g) =  0;
    dfde(3,g) =  1;
end

% Initialize matrices
M = zeros(NN,NN);
A = zeros(NN,NN);
% no need for load vector b since no source term
```



FEM Code

```
for e = 1:NE
    for g = 1:NG
        dxds = 0; dxde = 0;
        dyds = 0; dyde = 0;
        for i = 1:NL
            ig = T(e,i);
            dxds = dxds + xp(ig)*dfds(i,g);
            dxde = dxde + xp(ig)*dfde(i,g);

            dyds = dyds + yp(ig)*dfds(i,g);
            dyde = dyde + yp(ig)*dfde(i,g);

        end
        J = dxds*dyde - dxde*dyds;
        detJ = abs(J);

        for il = 1:NL

            ig = T(e,il);
            dfidx = ( dyde*dfds(il,g) - dyds*dfde(il,g) ) / J;
            dfidy = (-dxde*dfds(il,g) + dxds*dfde(il,g) ) / J;

            for jl = 1:NL

                jg = T(e,jl);
```

```
                dfjdx = ( dyde*dfds(jl,g) - dyds*dfde(jl,g) ) / J;
                dfjdy = (-dxde*dfds(jl,g) + dxds*dfde(jl,g) ) / J;

                % Mass matrix
                M(ig,jg) = M(ig,jg) + f(il,g)*f(jl,g)*gw(g)*detJ;

                % Stiffness matrix
                A(ig,jg) = A(ig,jg) + D*(dfidx*dfjdx +
dfidy*dfjdy)*gw(g)*detJ;

            end
        end
    end
end

% Identify boundaries from DB matrix
topNodes = find(Dirichlet == 1);
botNodes = find(Dirichlet == 2);

% Initial concentration vector
cInitial = zeros(NN,1);

cInitial(topNodes) = c0;
cInitial(botNodes) = 0;

% ODE solve using ode15s
tspan = [0 tFinal];

odefun = @(t,c) DrugDiffusionODE_Full(t,c,M,A,Dirichlet,c0,k); %solve for c
from dcdt eqn

opts = odeset('RelTol',1e-5, 'AbsTol',1e-8);

[tSol,cSol] = ode15s(odefun,tspan,cInitial,opts);

% Extract concentration fields
cInitialPlot = cSol(1,:)';
cFinal = cSol(end,:)';

% FEM half-life using top boundary concentration
topConcTime = zeros(length(tSol),1);

for q = 1:length(tSol)
    cNow = cSol(q,:)';
    topConcTime(q) = mean(cNow(topNodes));
end

% Half concentration for half life
halfConc = c0/2;

% Find FEM half-life, first instance of half of c0
idxHalf = find(topConcTime <= halfConc, 1, 'first');

if isempty(idxHalf)
```

find first instance of $c = \frac{1}{2}c_0$

```
    femHalfLife = NaN;
else
    if idxHalf == 1
        femHalfLife = tSol(1);
    else
        % Linear interpolation
        t1 = tSol(idxHalf-1);
        t2 = tSol(idxHalf);

        c1 = topConcTime(idxHalf-1);
        c2 = topConcTime(idxHalf);

        femHalfLife = t1 + (halfConc - c1)*(t2 - t1)/(c2 - c1);
    end
end

% Analytical half-life
analyticalHalfLife = log(2)/k;
```

find the 2 time points that surround the half life $t_{1/2}$

(t_1, c_1) (t_2, c_2)

Comparison of analytical and FEM values

```
fprintf('Initial top concentration = %.6g\n', c0);
fprintf('Half concentration = %.6g\n', halfConc);
fprintf('FEM half-life from top boundary = %.6g hr\n', femHalfLife);
fprintf('Analytical half-life = %.6g hr\n', analyticalHalfLife);

if ~isnan(femHalfLife)

    percentDifference = abs(femHalfLife - analyticalHalfLife) /
analyticalHalfLife * 100;
    fprintf('Percent difference = %.6g %%\n', percentDifference);

else
    fprintf('Percent difference = FEM half-life not reached\n');
end
```

```
Initial top concentration = 0.04
Half concentration = 0.02
FEM half-life from top boundary = 66.0679 hr
Analytical half-life = 66.014 hr
Percent difference = 0.0816384 %
```

Plots

Plot concentration at t = 0

```
figure;
trisurf(T,xp,yp,cInitialPlot);
shading interp;
colorbar;
title('Drug Concentration at t = 0');
xlabel('x (mm)');
ylabel('y (mm)');
```

```
zlabel('c(x,y,0)');
view(3);

% Plot concentration at final time
figure;
trisurf(T,xp,yp,cFinal);
shading interp;
colorbar;
title(sprintf('Drug Concentration at t = %.2f hr',tSol(end)));
xlabel('x (mm)');
ylabel('y (mm)');
zlabel('c(x,y,t) (mg)');
view(3);

% Plot final concentration heat map
figure;
trisurf(T,xp,yp,cFinal);
shading interp;
colorbar;
axis equal;
view(2);
title(sprintf('Drug Concentration at t = %.2f hr',tSol(end)));
xlabel('x (mm)');
ylabel('y (mm)');

% Plot concentration at the top boundary over time
figure;
plot(tSol, topConcTime, 'LineWidth', 2);
hold on;

% Half concentration line
yline(c0/2, '--r', 'Half Concentration', 'LineWidth', 1.5);

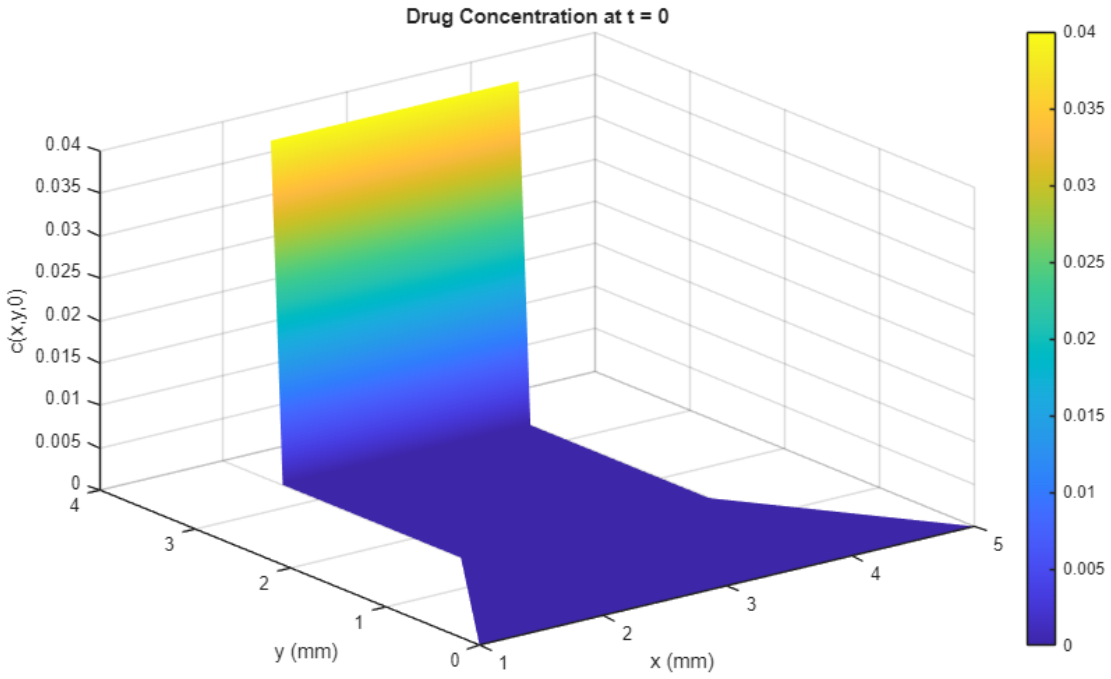
xlabel('Time (hr)');
ylabel('Top Boundary Concentration (mg)');
title('Top Boundary Concentration vs Time');
grid on;
set(gca, 'FontSize', 12);

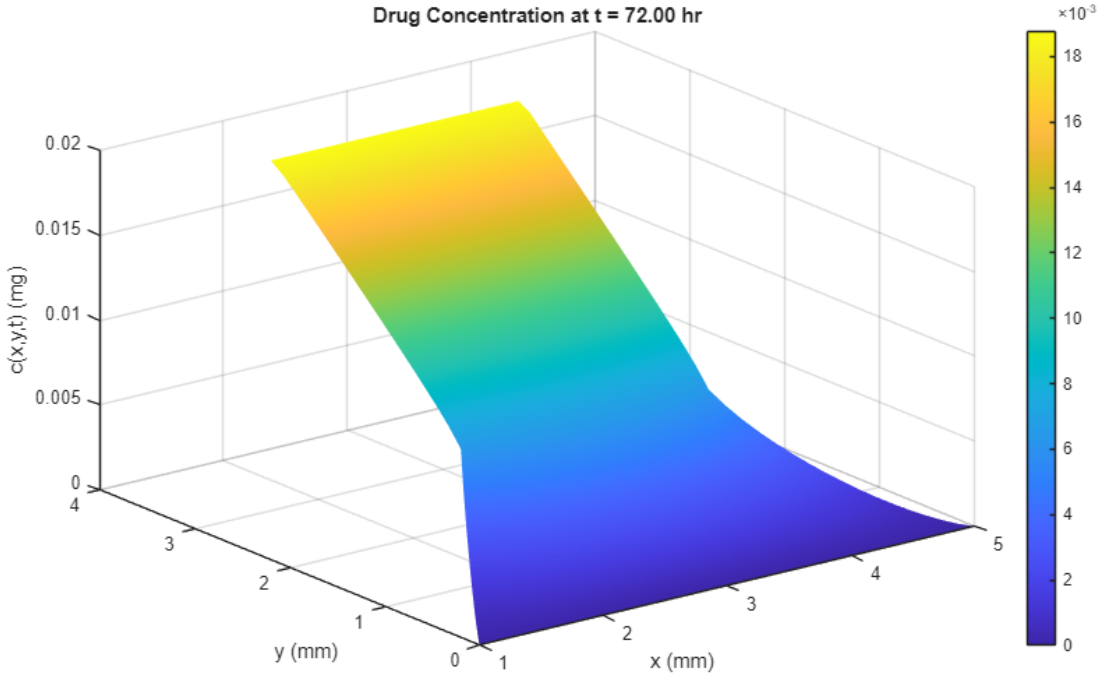

% ODE function
function dcdt = DrugDiffusionODE_Full(t,c,M,A,Dirichlet,c0,k)
    NN = length(c);
    % FEM ODE: M dc/dt + A c = 0

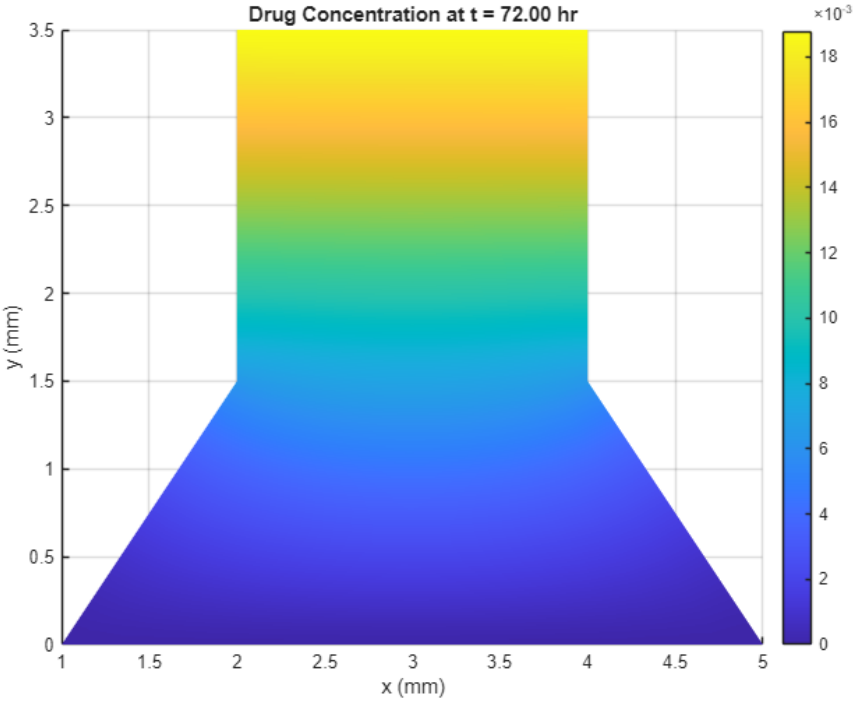
    dcdt = M \ (-A*c);

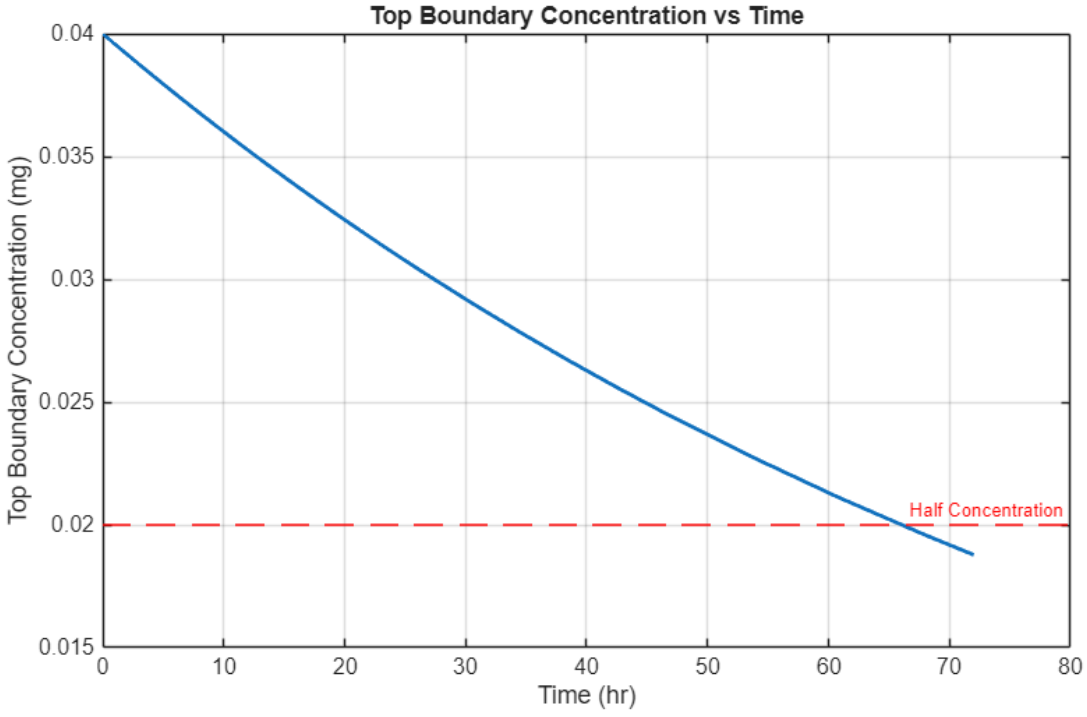
    % Enforce boundary conditions
    cTop = c0*exp(-k*t);
    for i = 1:NN
        if Dirichlet(i) == 1 % Top boundary
            dcdt(i) = -k*cTop;
        elseif Dirichlet(i) == 2 % Bottom boundary
            dcdt(i) = 0;
```

```
        end
    end
end
```


Drug Concentration at t = 0
c(x,y,0)
0.04
0.035
0.03
0.025
0.02
0.015
0.01
0.005
0
y (mm)
4
3
2
1
0
x (mm)
1
2
3
4
5
0.04
0.035
0.03
0.025
0.02
0.015
0.01
0.005
0


Drug Concentration at t = 72.00 hr
c(x,y,t) (mg)
0.02
0.015
0.01
0.005
0
y (mm)
4
3
2
1
0
x (mm)
1
2
3
4
5
×10-3
18
16
14
12
10
8
6
4
2
0





Published with MATLAB® R2026a

Table of Contents

..... 1

```
function [T, xp, yp, Dirichlet] = MyMesher(px, py, NNT, DB)

% function MyMesher generates a mesh inside a polygon defined by the
% vectors of points px and py. For example, a unit square consisting of the
% points (0,0), (1,0), (1,1), and (0,1) connected in that order would have
% px = [0; 1; 1; 0] and py = [0; 0; 1; 1].
% The scalar NNT is the number of target number of points in the interior
% of the mesh.

[T,X] = poly2fv(px,py); % Convert polygon to triangles

NE = length(T(:,1));
NN = length(X(:,1));
NEnew = NE;
Xnew = X;

while NN < (0.8 * NNT) %keep refining mesh until nodes reaches 80% of NNT
(in this case NNT is 500)
    Tnew = [T;zeros(3*length(T(:,1)),3)];
    X = Xnew;
    NE = NEnew;
    for e = 1:NE
        % set the nodes for each of the triangles
        n4 = T(e,1);
        n5 = T(e,2);
        n6 = T(e,3);
        %compute midpoints of each triangle edges
        p1 = (X(T(e,1),:)+X(T(e,2),:))/2;
        p2 = (X(T(e,3),:)+X(T(e,2),:))/2;
        p3 = (X(T(e,1),:)+X(T(e,3),:))/2;
        % check if midpoint already exists
        u = find((abs(Xnew(:,1)-p1(1))+ abs(Xnew(:,2)-p1(2)))< 1e-6);
        if length(u) > 0 % if the midpoint already exists, use the same node
            n1 = u(1);
        else
            n1 = NN + 1; % if it doesn't exist, create a new node
            NN = n1;
            Xnew(NN,:) = p1;
        end
        %repeat for the other two nodes of the triangle
        u = find((abs(Xnew(:,1)-p2(1))+ abs(Xnew(:,2)-p2(2)))< 1e-6);
        if length(u) > 0
            n2 = u(1);
        else
            n2 = NN + 1;
            NN = n2;
            Xnew(NN,:) = p2;
```

```
        end
        %repeat
        u = find((abs(Xnew(:,1)-p3(1))+ abs(Xnew(:,2)-p3(2)))< 1e-6);
        if length(u) > 0
            n3 = u(1);
        else
            n3 = NN + 1;
            NN = n3;
           Xnew(NN,:) = p3;
        end
        % map new triangles
        T(e,:) = [n1, n2, n3];
        T(NEnew+1,:) = [n4, n1, n3];
        T(NEnew+2,:) = [n5, n2, n1];
        T(NEnew+3,:) = [n6, n3, n2];
        NEnew = NEnew + 3;
    end
end
% final physical coordinates in x and y
xp = Xnew(:,1);
yp = Xnew(:,2);

% initialize dierchlet boundary vector
Dirichlet = zeros(NN,1);

% DB format: [boundary ID, x1 y1 x2 y2] where (x1, y1) and (x2, y2) make
% the line segment that forms the boundary
for k = 1:NN %for all nodes
    for b = 1:length(DB(:,1)) %loops through all boundary segments
        % check if node is on the boundary
        [i,o] = inpolygon(Xnew(k,1),Xnew(k,2),...
            [DB(b,1) DB(b,3)],[DB(b,2) DB(b,4)]);
        if o == 1
            Dirichlet(k,1) = b; %and add it to BC vector if it is
        end
    end
end
```

```
Not enough input arguments.

Error in MyMesher (line 10)
[T,X] = poly2fv(px,py); % Convert polygon to triangles
                ^^
```

Published with MATLAB® R2026a